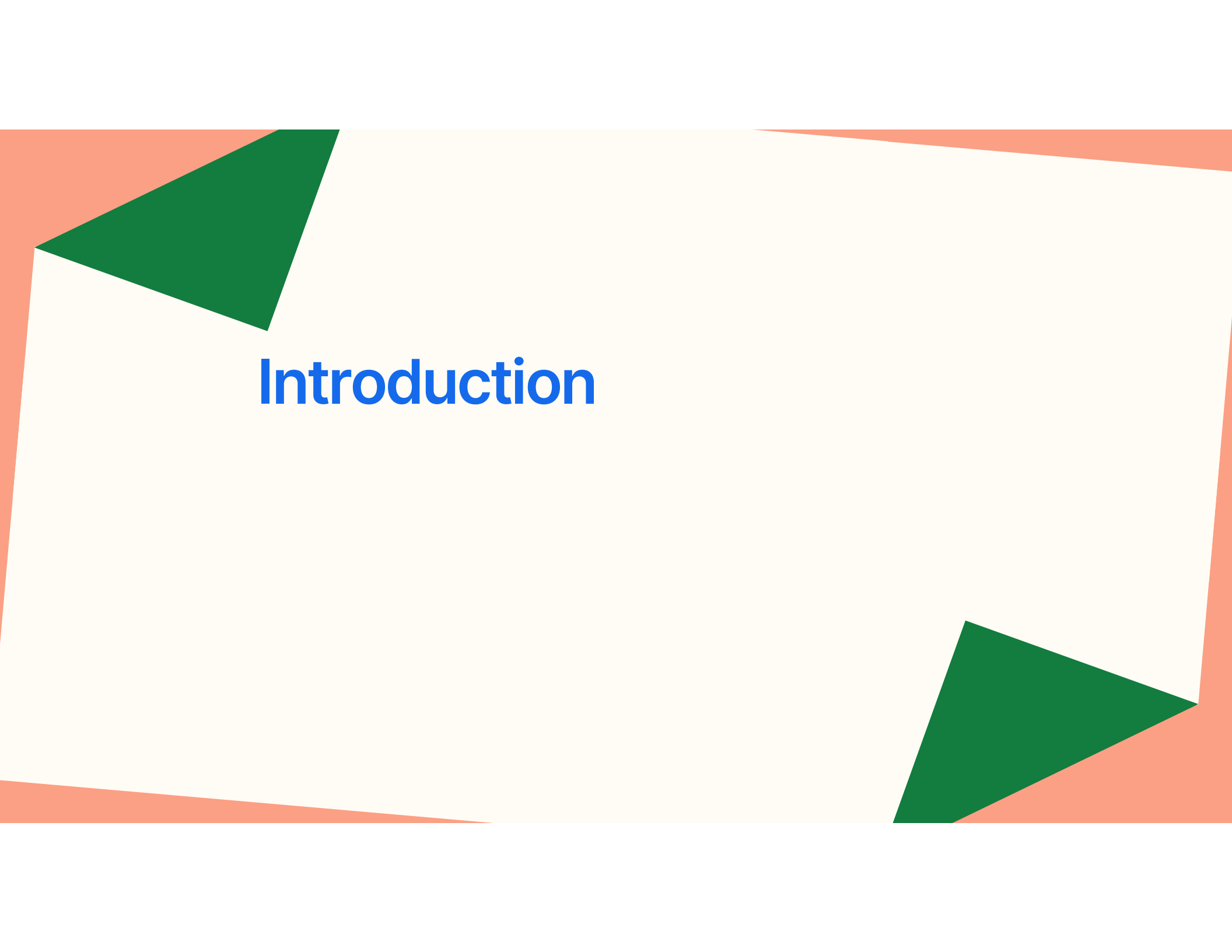


The background features a central cream-colored rectangle. This rectangle is framed by orange shapes at the top, bottom, and sides. Two dark green triangles are positioned in the upper-left and lower-right corners of the cream rectangle.

MoodyClues

Final Submission

Team 3

The background features a large, irregular white shape in the center, surrounded by orange-colored areas. Two dark green triangles are positioned on the left and right sides of the white shape, pointing towards the center.

Introduction

Introduction

There is a prevalence and rising trend in poor mental health in Singaporeans.

Almost 1 in 3 people between 15 and 35 years of age have reported "severe or extremely severe symptoms of depression, anxiety and/or stress."

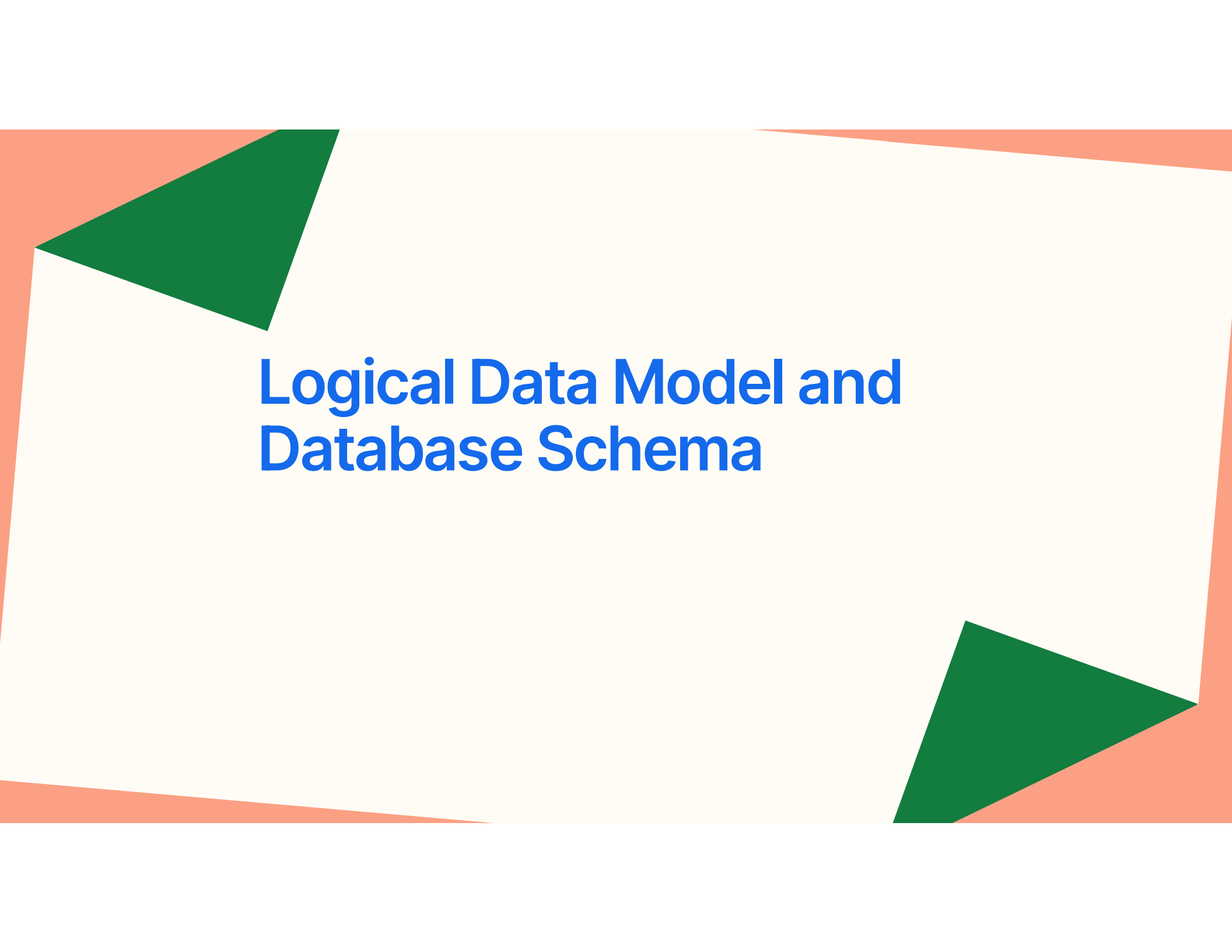
Research on journaling has found that it can help manage mental health symptoms.

But what if journals could be used to monitor the emotional well-being of those that might be vulnerable or those that are unable to communicate easily?

Introducing

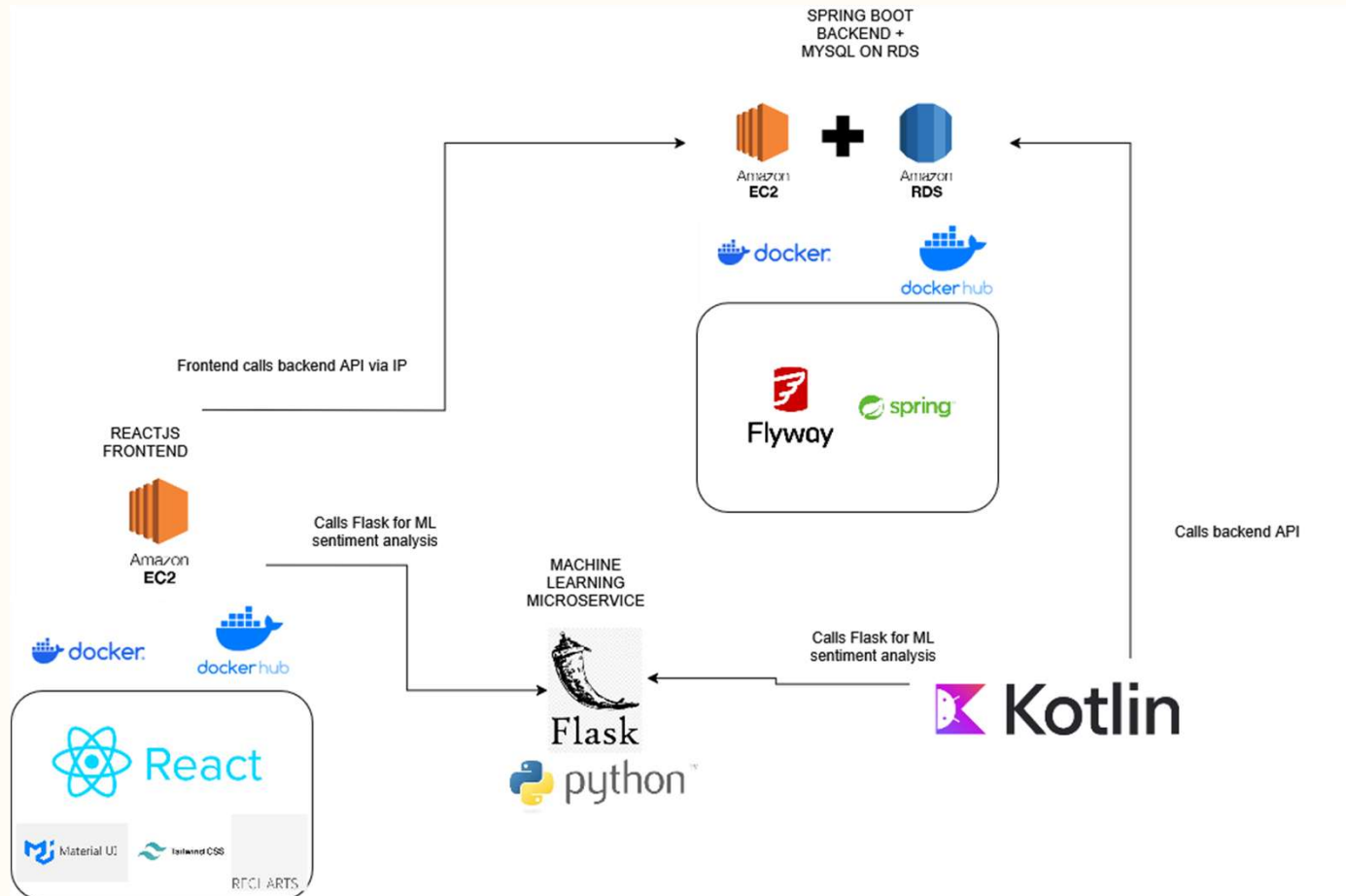
MoodyClues

for Journaling and Emotional Well-being Monitoring

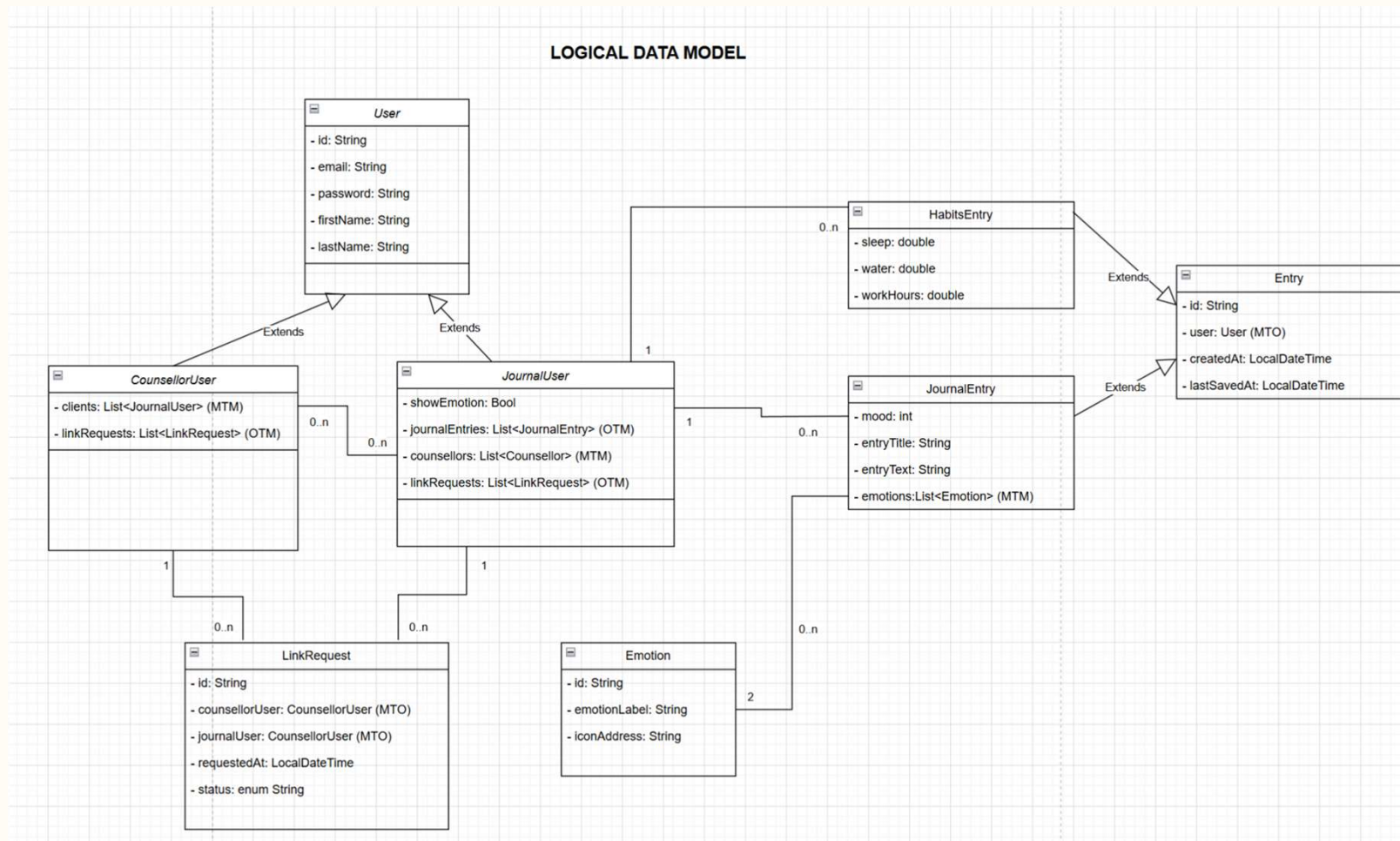
The background features a central light cream-colored rectangle. This rectangle is framed by orange shapes: a thin border at the top and bottom, and larger triangular shapes at the corners. Specifically, there is a large green triangle in the top-left corner and another in the bottom-right corner, both pointing towards the center. The overall design is minimalist and modern.

Logical Data Model and Database Schema

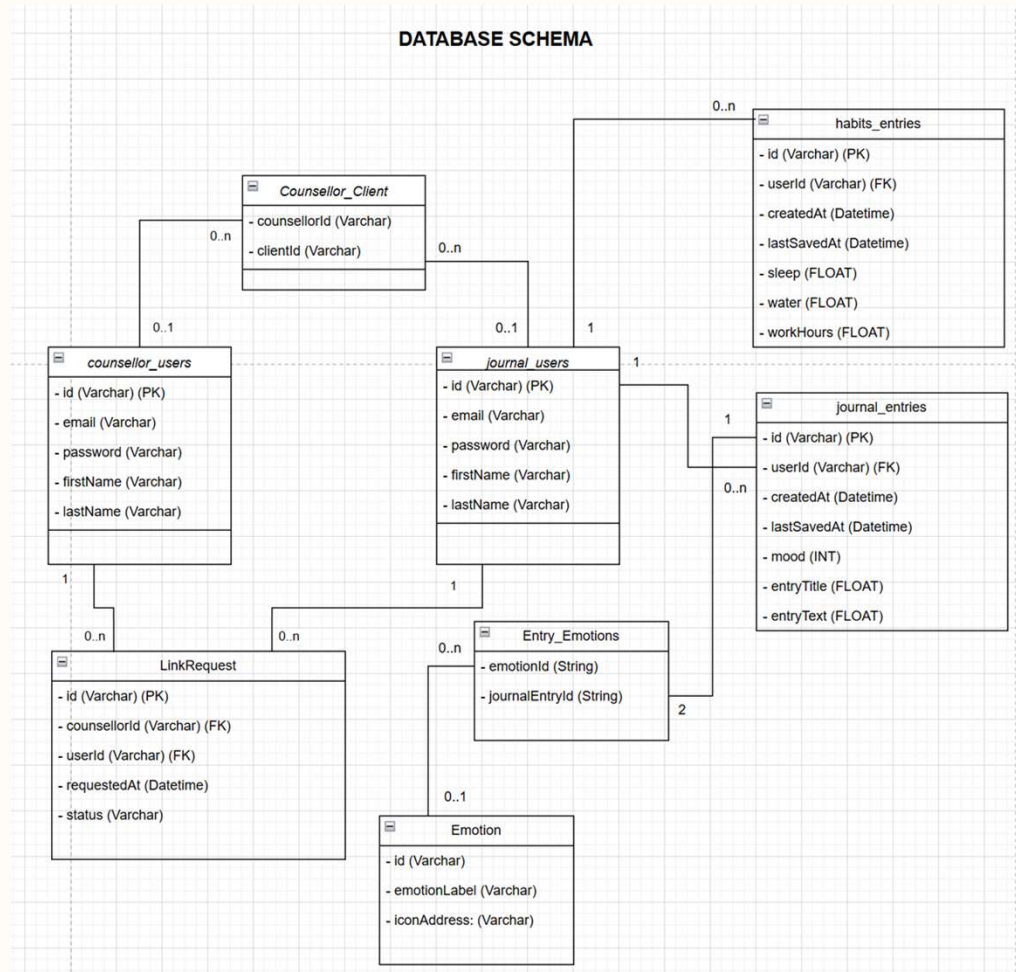
Application and Deployment Architecture



Logical Data Model

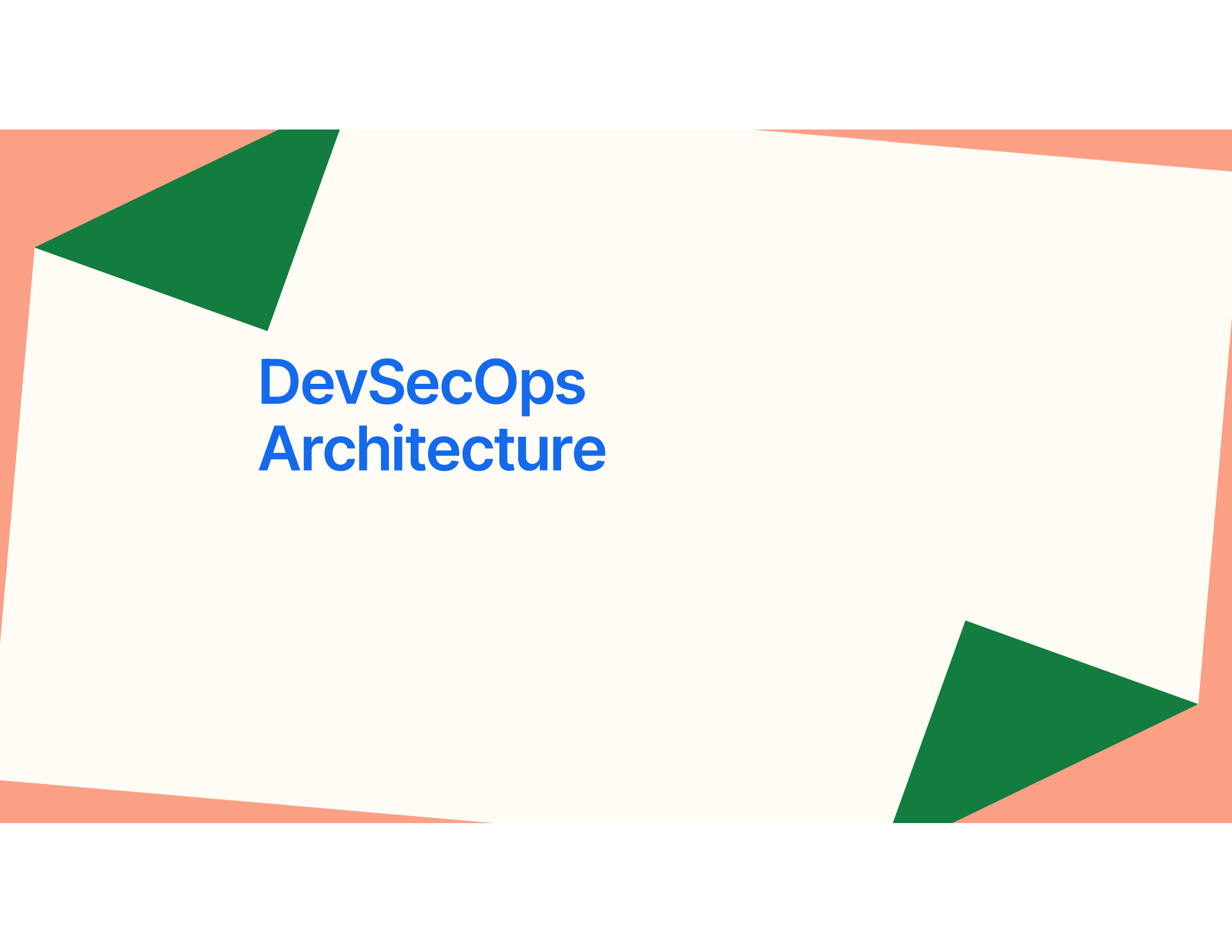


Database Schema



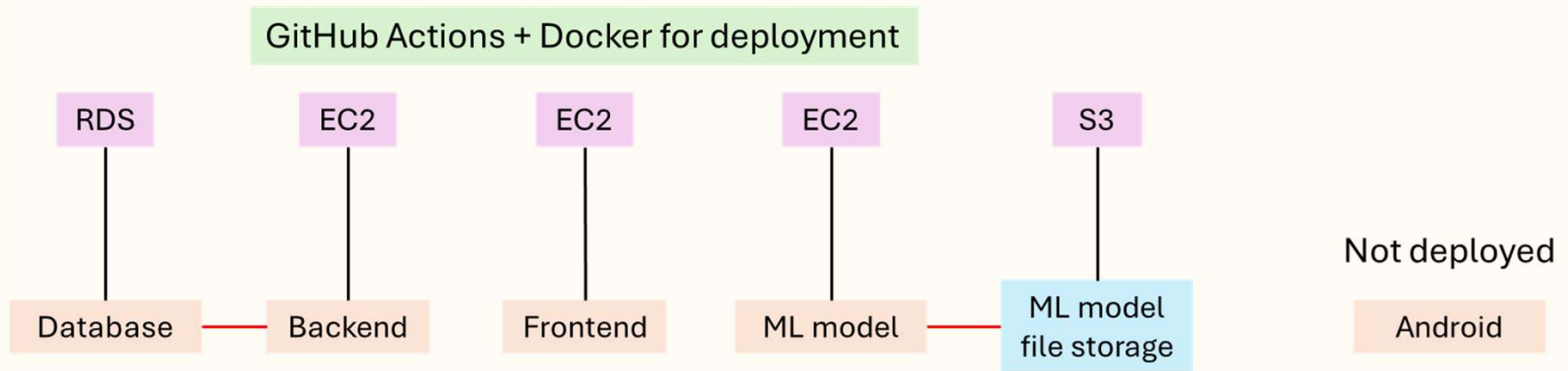
Application Main Deliverables

Feature / Deliverable	Member in-charge
- Web mock screens (All) - Spring Boot Backend (All code + architecture planning + API testing & documentation) - ReactJS (Login, Register, Home, User + Counsellor Dashboard Screens) - Backend + Database + Frontend Deployment (AWS EC2 + RDS)	Dion
- Machine learning model for sentiment analysis (Python) - ML model deployment and CI/CD pipeline implementation (FlaskAPI, AWS EC2 + S3, GitHub Actions) - ReactJS (User screens)	Hiroyo
- Android journal users' screens (Login, Register, Home, Notification, Emotion setting) - Android journal users' screens (Login, Register, Home, Invitation) - Android call backend (All)	Ma Li
- Android Counsellor functionality	Thina
- ReactJS - Counsellor screens	Shengyi



DevSecOps Architecture

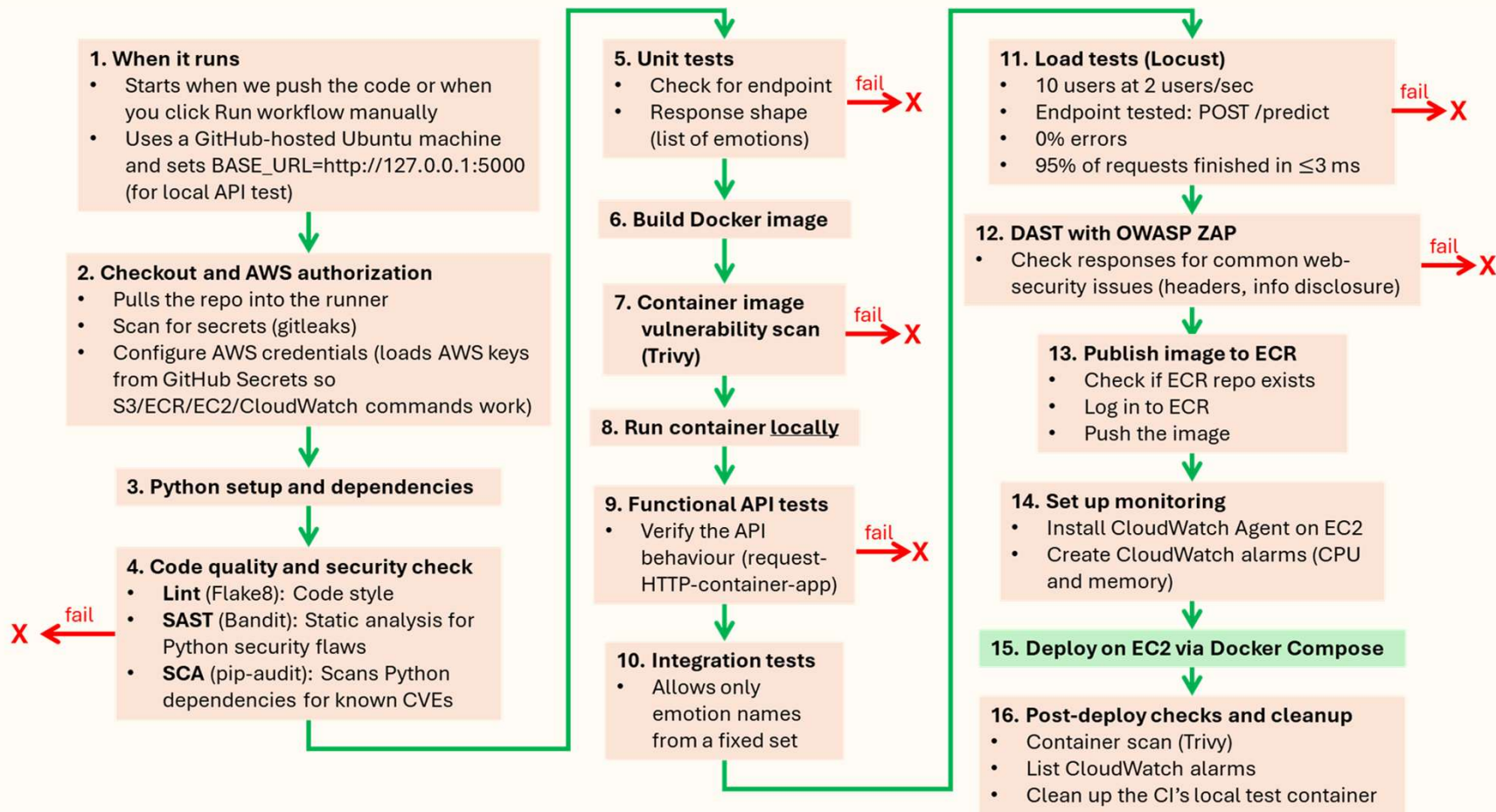
Deployment to AWS



AWS Services

- EC2 Host backend, frontend, and ML model
- S3 Store large ML model files
- RDS Database (MySQL)
- ECR Container image registry
- IAM Role Give EC2 permission to pull from ECR and S3
- CloudWatch Logs and monitoring

Machine Learning Model CI/CD



Machine Learning Model CI/CD

GitHub Actions

The screenshot displays the GitHub Actions interface for a workflow run titled "Added Flask-Cors for frontend (3) #157". The left sidebar shows the workflow status as "Completed" and the job "build-and-deploy" as "Succeeded". The main panel shows the job details, including a list of steps and their status. The job "build-and-deploy" succeeded 2 days ago in 12m 58s. The steps listed are:

- Set up job
- Checkout code
- Scan for secrets (gitLeaks)
- Sanity echo
- Configure AWS credentials
- Download model & thresholds from S3
- Verify downloaded files
- Set up Python
- Install Python dependencies
- Lint with flake8
- Bandit security scan
- pip-audit dependency scan
- Run unit tests
- Build Docker image (local tag)
- Show app.py inside the built image
- Smoke-test torch/transformers imports inside image
- Free up space for Trivy
- Scan Docker image with Trivy
- Define container name
- Clean any old test container
- Start ML model container (stub mode for CI)
- Show container status and logs (pre-sanity)
- Wait for API (HTTP 200 on /)
- Functional API tests
- Integration tests (if present)
- Skip integration tests (none found)
- Load tests with Locust (if present)
- Skip load tests (none found)
- Prepare ZAP reports dir (fix perms)
- ZAP baseline (docker, save reports)
- Show ZAP outputs
- Upload ZAP reports
- Ensure ECR repo exists
- Login to Amazon ECR
- Tag & push image to ECR (staging + sha + latest)
- Ensure EC2 is running
- Wait for SSH on EC2
- Install/Update CloudWatch Agent on EC2
- Create/Ensure CloudWatch Alarms (CPU & Memory)
- Deploy to EC2 via Docker Compose (v2)
- Post-deploy Trivy scan on EC2 (optional)
- Verify CloudWatch alarms
- Show container logs on failure
- Cleanup test container
- Post Scan Docker image with Trivy
- Post Set up Python
- Post Configure AWS credentials
- Post Checkout code
- Complete job

- Note that Terraform and Ansible were not used because for the scale of our project, we do not make use of them very much
- Instead of staging, functional and integration tests were done locally

Machine Learning Model CI/CD

Lint (flake8)

```

1  ▶ Run pip install flake8
16 Collecting flake8
17   Downloading flake8-7.3.0-py2.py3-none-any.whl.metadata (3.8 kB)
18 Collecting mccabe<0.8.0,>=0.7.0 (from flake8)
19   Downloading mccabe-0.7.0-py2.py3-none-any.whl.metadata (5.0 kB)
20 Collecting pycodestyle<2.15.0,>=2.14.0 (from flake8)
21   Downloading pycodestyle-2.14.0-py2.py3-none-any.whl.metadata (4.5 kB)
22 Collecting pyflakes<3.5.0,>=3.4.0 (from flake8)
23   Downloading pyflakes-3.4.0-py2.py3-none-any.whl.metadata (3.5 kB)
24 Downloading flake8-7.3.0-py2.py3-none-any.whl (57 kB)
25 Downloading mccabe-0.7.0-py2.py3-none-any.whl (7.3 kB)
26 Downloading pycodestyle-2.14.0-py2.py3-none-any.whl (31 kB)
27 Downloading pyflakes-3.4.0-py2.py3-none-any.whl (63 kB)
28 Installing collected packages: pyflakes, pycodestyle, mccabe, flake8
29
30 Successfully installed flake8-7.3.0 mccabe-0.7.0 pycodestyle-2.14.0 pyflakes-3.4.0
31 ml-model/app.py:82:1: F841 local variable 'e' is assigned to but never used
32 ml-model/app.py:93:1: F841 local variable 'e' is assigned to but never used
33 ml-model/app.py:105:1: F841 local variable 'e' is assigned to but never used
34 Error: Process completed with exit code 1.
```



```

1  ▶ Run pip install flake8
16 Collecting flake8
17   Downloading flake8-7.3.0-py2.py3-none-any.whl.metadata (3.8 kB)
18 Collecting mccabe<0.8.0,>=0.7.0 (from flake8)
19   Downloading mccabe-0.7.0-py2.py3-none-any.whl.metadata (5.0 kB)
20 Collecting pycodestyle<2.15.0,>=2.14.0 (from flake8)
21   Downloading pycodestyle-2.14.0-py2.py3-none-any.whl.metadata (4.5 kB)
22 Collecting pyflakes<3.5.0,>=3.4.0 (from flake8)
23   Downloading pyflakes-3.4.0-py2.py3-none-any.whl.metadata (3.5 kB)
24 Downloading flake8-7.3.0-py2.py3-none-any.whl (57 kB)
25 Downloading mccabe-0.7.0-py2.py3-none-any.whl (7.3 kB)
26 Downloading pycodestyle-2.14.0-py2.py3-none-any.whl (31 kB)
27 Downloading pyflakes-3.4.0-py2.py3-none-any.whl (63 kB)
28 Installing collected packages: pyflakes, pycodestyle, mccabe, flake8
29
30 Successfully installed flake8-7.3.0 mccabe-0.7.0 pycodestyle-2.14.0 pyflakes-3.4.0
```

- Lint flagged the local variable that is never used
- Fixed by removing that variable in the code

Machine Learning Model CI/CD

SAST (Bandit)

```
97 -----
98 >> Issue: [B615:huggingface_unsafe_download] Unsafe Hugging Face Hub download without revision pinning in from_pretrained()
99 Severity: Medium Confidence: High
100 CWE: CWE-494 (https://cwe.mitre.org/data/definitions/494.html)
101 More Info: https://bandit.readthedocs.io/en/1.8.6/plugins/b615\_huggingface\_unsafe\_download.html
102 Location: ml-model/ml_model_code.py:168:8
103 167 # Load DeBERTa model for multi-label classification
104 168 model = AutoModelForSequenceClassification.from_pretrained(
105 169     'microsoft/deberta-v3-small', num_labels=8, problem_type='multi_label_classification'
106 170 )
107 171
108 -----
109 >> Issue: [B614:pytorch_load] Use of unsafe PyTorch load
110 Severity: Medium Confidence: High
111 CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
112 More Info: https://bandit.readthedocs.io/en/1.8.6/plugins/b614\_pytorch\_load.html
113 Location: ml-model/ml_model_code.py:274:26
114 273 if os.path.exists(best_model_path):
115 274     model.load_state_dict(torch.load(best_model_path, map_location=device))
116 275     model.to(device)
117
118 -----
119
```

```
119 -----
120
121 Code scanned:
122     Total lines of code: 268
123     Total lines skipped (#nosec): 0
124     Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0
125
126 Run metrics:
127     Total issues (by severity):
128         Undefined: 0
129         Low: 0
130         Medium: 7
131         High: 0
132     Total issues (by confidence):
133         Undefined: 0
134         Low: 0
135         Medium: 1
136         High: 6
137 Files skipped (0):
138 Error: Process completed with exit code 1.
```

- SAST scan flagged some insecure coding practices
- It is acceptable to suppress these because
 - The model file is produced by my own training job
 - Stored in AWS S3 with access control and fetched when building the image, not during runtime

Machine Learning Model CI/CD

DAST (OWASP ZAP)



ZAP Scanning Report

Site: <http://127.0.0.1:5000>

Generated on Sat, 9 Aug 2025 14:33:05

ZAP Version: 2.16.1

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	4
Informational	1
False Positives:	0

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

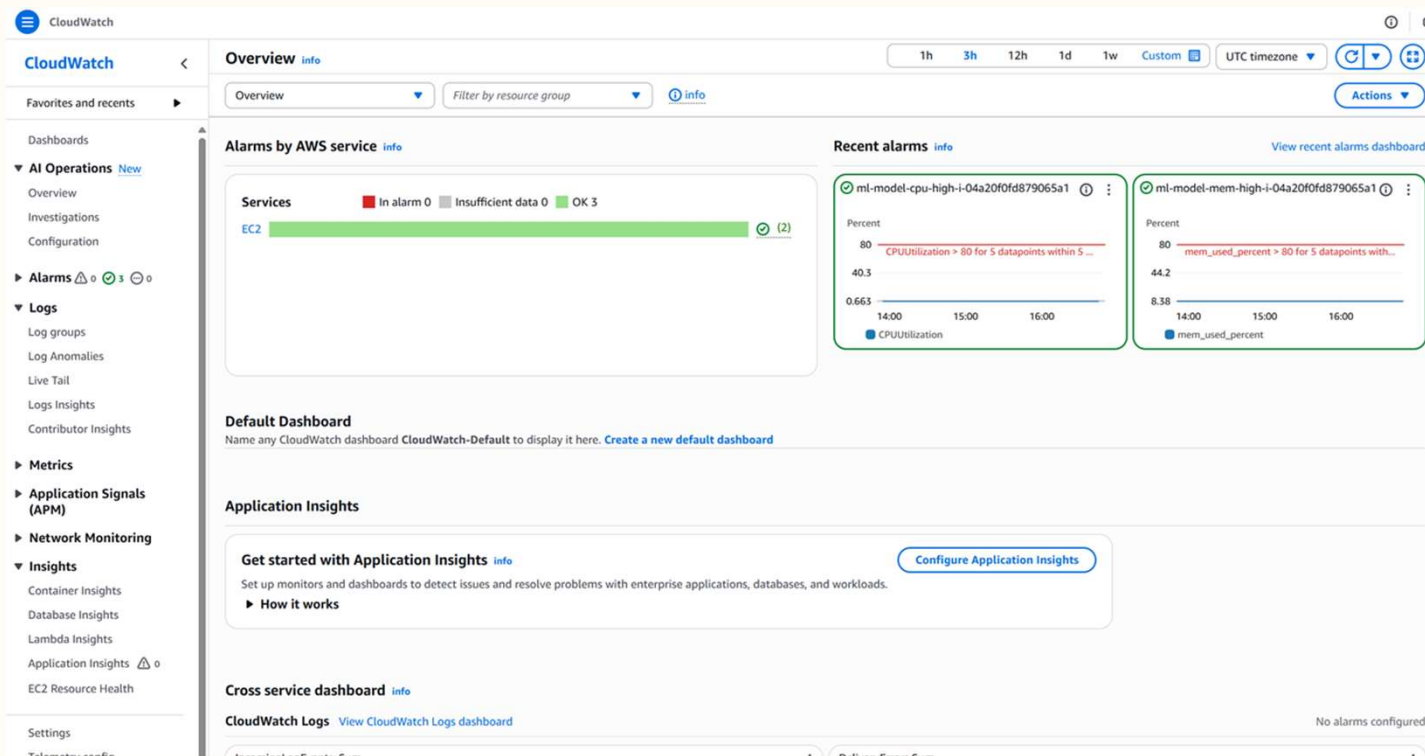
Alerts

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	3
Missing Anti-clickjacking Header	Medium	1
Insufficient Site Isolation Against Spectre Vulnerability	Low	3
Permissions Policy Header Not Set	Low	3
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	3
X-Content-Type-Options Header Missing	Low	1
Storable and Cacheable Content	Informational	3

- DAST scan showed some alerts, but none were high risk

Machine Learning Model CI/CD

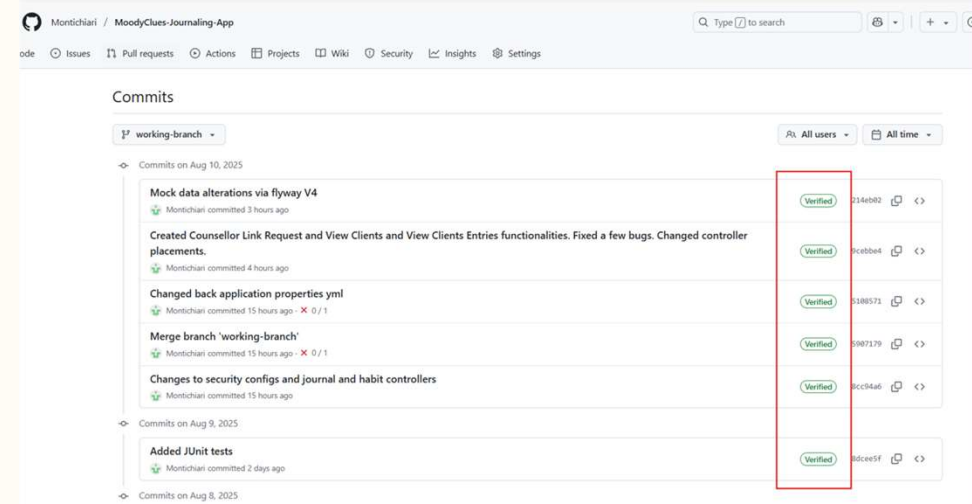
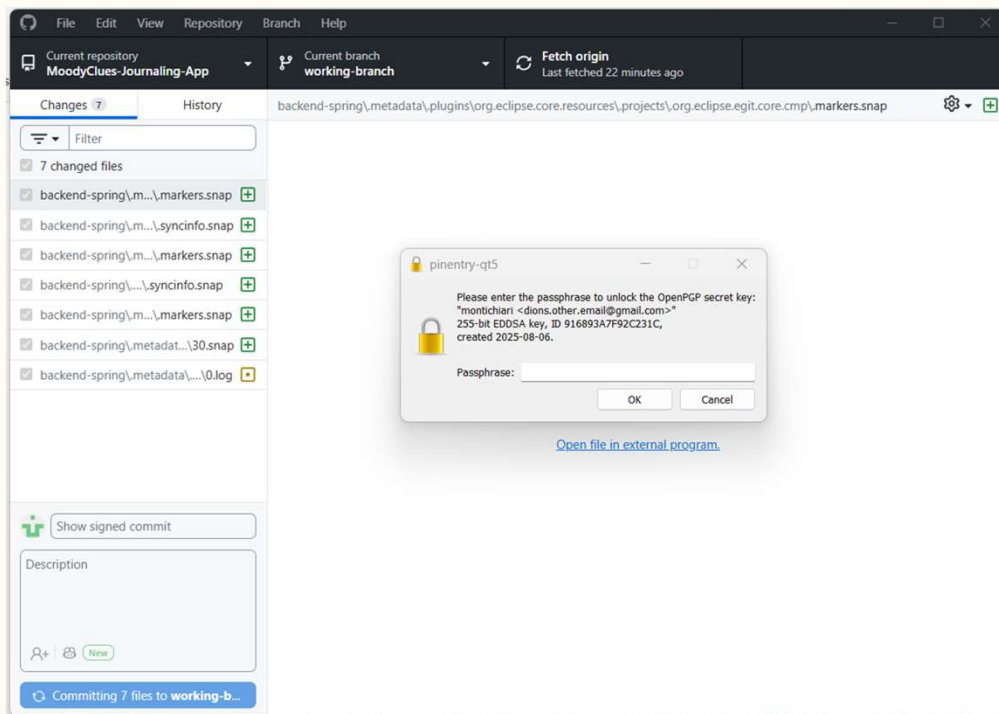
AWS CloudWatch



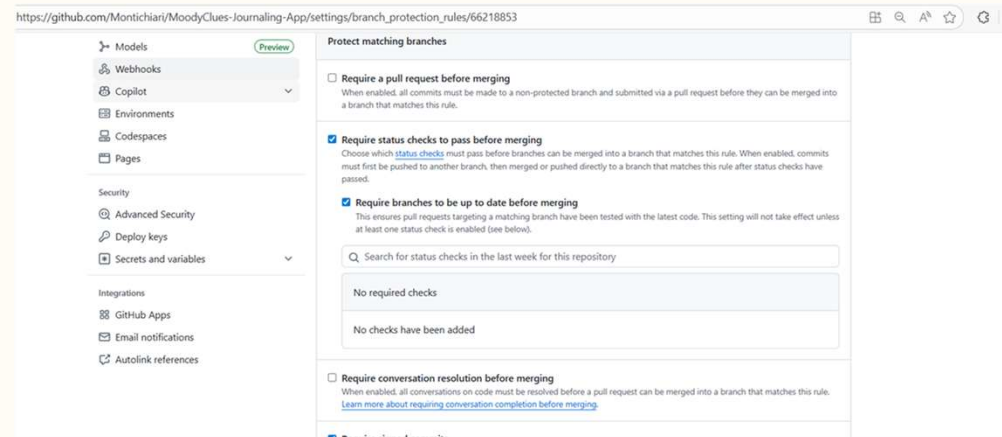
- Set up CloudWatch for post-deployment monitoring
- Alerts when CPU or memory utilization exceeds 80%

Backend CI/CD

GitHub Signed Commits



Branch Protection



Backend CI/CD

Secrets Protection

19

The screenshot shows the GitHub Actions secrets management interface for the repository `Montichiari/MoodyClues-Journaling-App`. The URL in the browser is `https://github.com/Montichiari/MoodyClues-Journaling-App/settings/secrets/actions`.

The left sidebar contains the following navigation items:

- Code and automation
 - Branches
 - Tags
 - Rules
 - Actions
 - Models
 - Webhooks
 - Copilot
 - Environments
 - Codespaces
 - Pages
- Security
 - Advanced Security
 - Deploy keys
 - Secrets and variables
- Integrations
 - GitHub Apps
 - Email notifications
 - Autolink references

The main content area is divided into two tabs: `Secrets` and `Variables`. The `Secrets` tab is active.

Under the `Environment secrets` section, there is a message: "This environment has no secrets." and a button labeled `Manage environment secrets`.

Under the `Repository secrets` section, there is a button labeled `New repository secret` and a table listing the repository secrets.

Name	Last updated
DOCKERHUB_TOKEN	2 days ago
DOCKERHUB_USERNAME	2 days ago
EC2_HOST	3 days ago
EC2_PUBLIC_IP	2 days ago
EC2_SSH_KEY	3 days ago
EC2_USER	3 days ago
RDS_JDBC_URL	2 days ago
RDS_PASSWORD	2 days ago
RDS_USERNAME	2 days ago

Backend CI/CD

JUnit Tests + Code coverage (JaCoCo)

The screenshot displays the Spring Tool Suite 4 IDE interface. The top-left pane shows the JUnit test results for `JournalUserServiceImpTest`, indicating that all 10 tests passed successfully. The main editor shows the source code of `JournalUserServiceImpTest.java`, which includes tests for finding users by email, handling missing users, and attempting login. The bottom-right pane shows the JaCoCo code coverage report.

Element	Coverage	Covered Instru...	Missed Instru...	Total Instru...
> EntryServiceImpl.java	0.0 %	0	206	206
> LinkRequestServiceImpl.java	0.0 %	0	150	150
> CustomUserDetailsServiceImpl.java	0.0 %	0	51	51
> JournalUserServiceImp.java	84.2 %	112	21	133
> com.moodyclues.controller	0.0 %	0	592	592
> com.moodyclues.model	18.8 %	71	307	378
> com.moodyclues.config	0.0 %	0	184	184

1 elements hidden by filter

Backend CI/CD

GitHub Actions Pipeline

GitHub / Actions

Upon signed commit and push to Main branch, starts the process by taking the code from the "backend-spring" folder.
Ensures only 1 deploy is run at a time.

Flyway Data Migration

GitHub Actions connects directly to database on RDS and runs Flyway.
Flyway:
Scans migration folder for versioned scripts ->
Checks/creates flyway_schema_history ->
Applies only the pending migrations.

Build, Dockerize, Push

The JAR is built, Docker logged into with secrets, image created and pushed to DockerHub.

Deploy on EC2

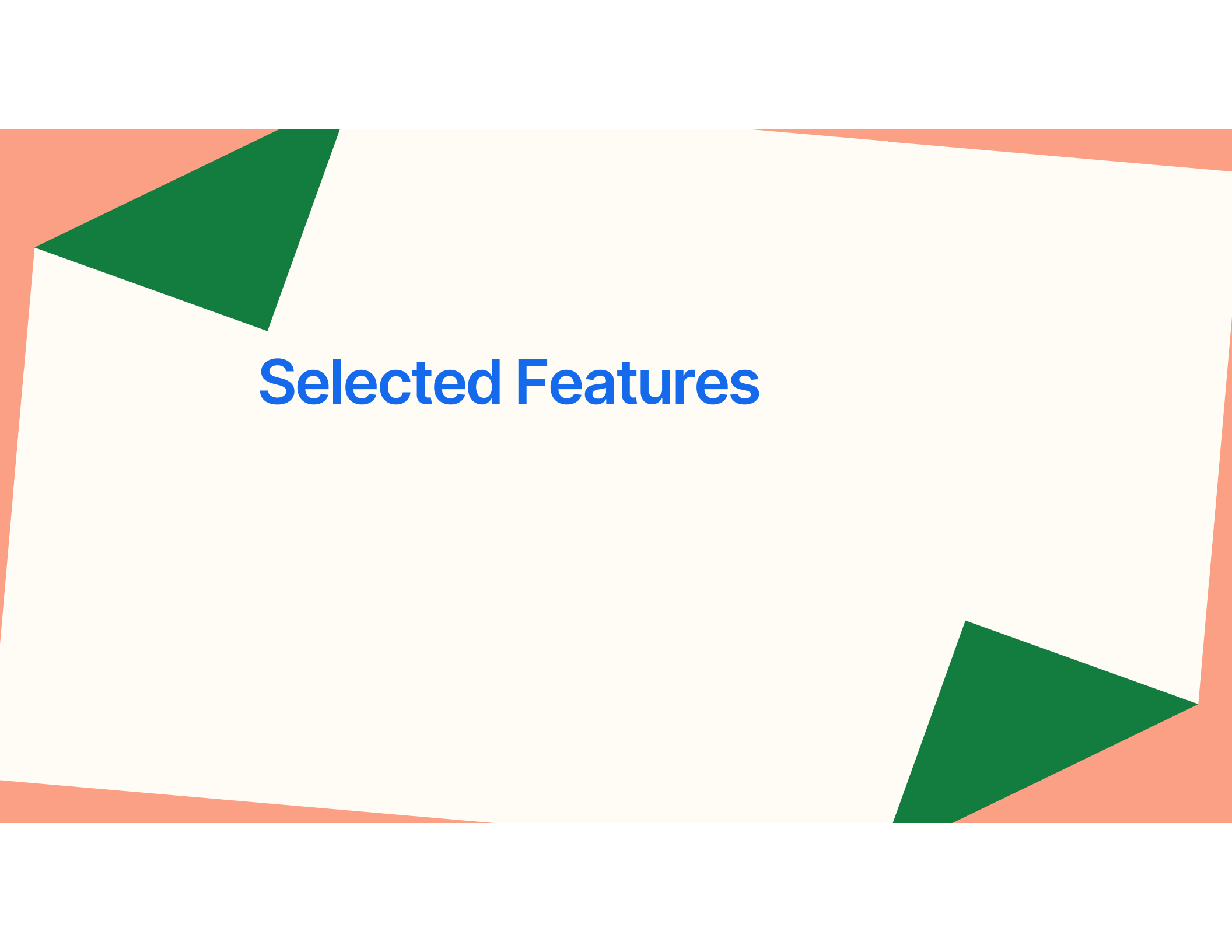
The script then SSH into the EC2 instance, stops any running container instance, pulls latest image from DockerHub, and runs the image.

Success

The backend is now updated with the latest changes / features automatically.

Backend CI/CD

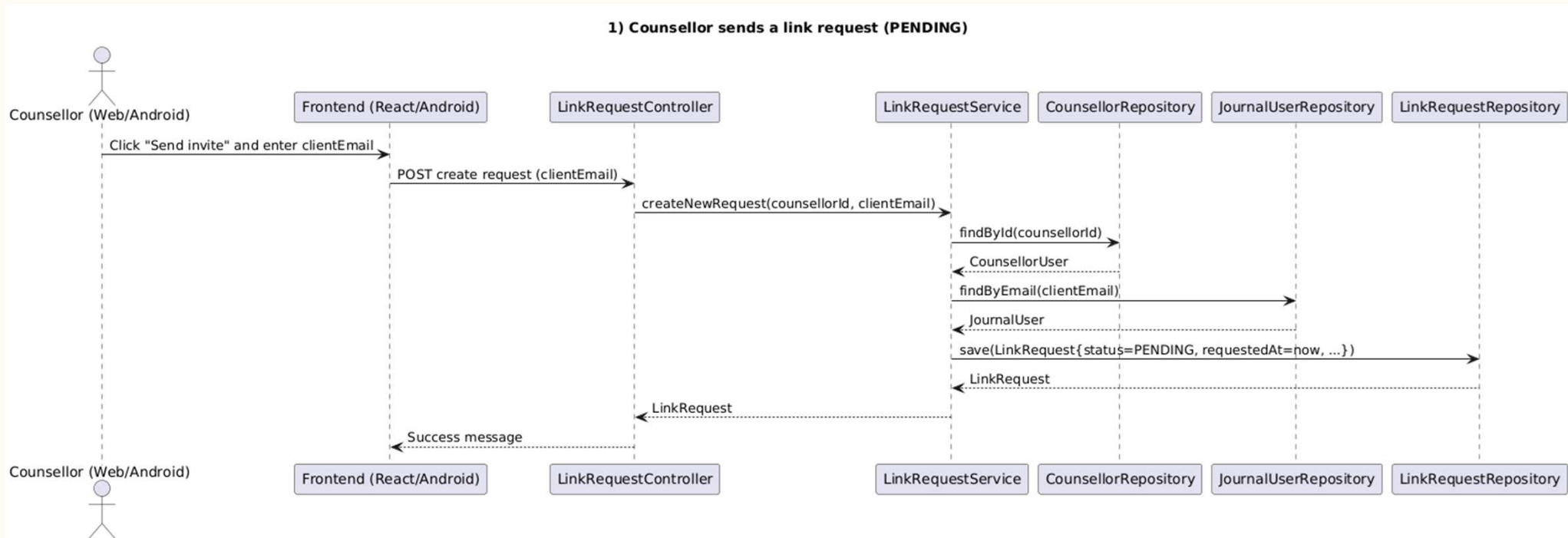
SAST

The background features a large, irregular, light cream-colored shape that resembles a piece of paper or a map. This shape is set against a solid light blue background. The cream shape has two dark green triangular sections cut out from its corners: one in the top-left and one in the bottom-right. The overall composition is minimalist and modern.

Selected Features

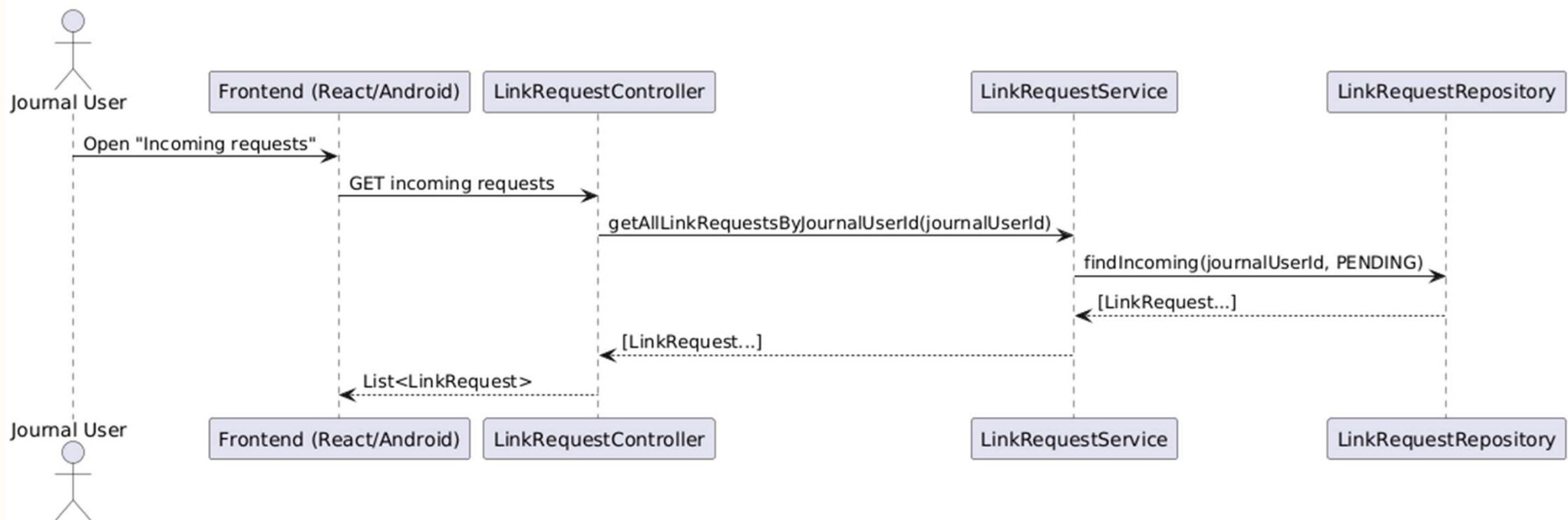
Feature Highlight #1 - Link Requests

Core feature needed to link Counsellor with JournalUser, to allow them to access their clients' entries, and have their projections in dashboard.

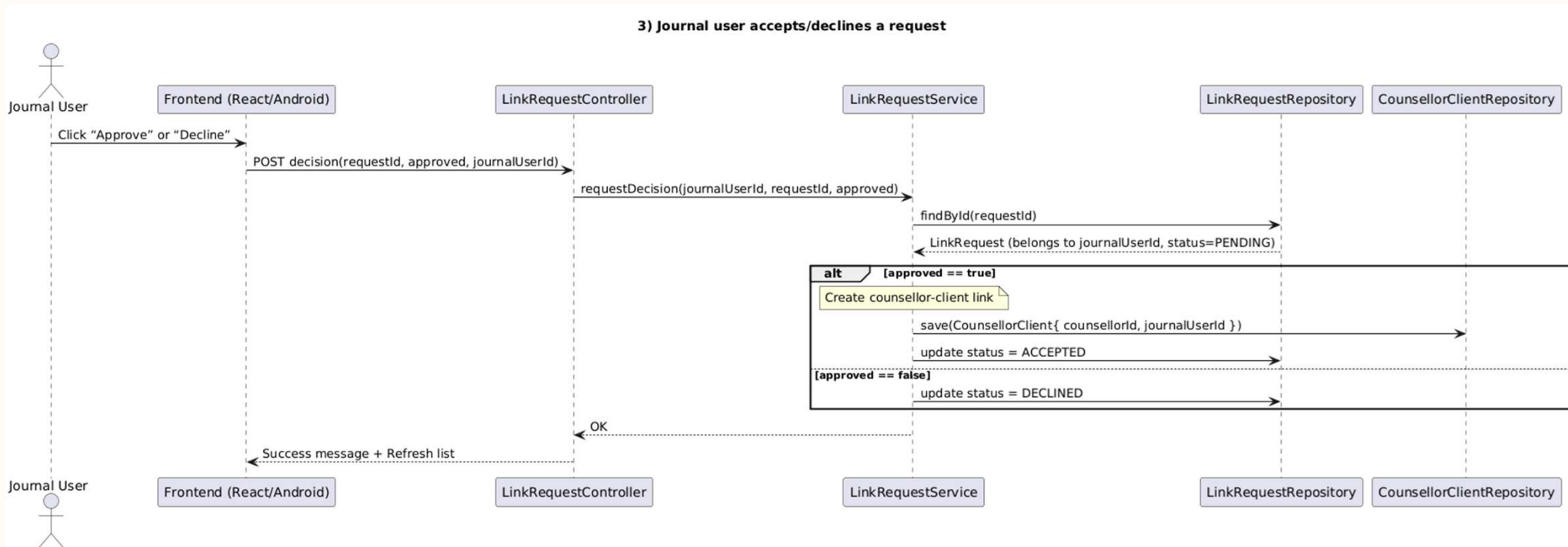


Feature Highlight #1 - Link Requests

2) Journal user views incoming link requests



Feature Highlight #1 - Link Requests



Feature Highlight #2 - Machine Learning

Machine Learning Model for Sentiment Analysis

Dataset card | Data Studio | Files and versions | xet | Community 7

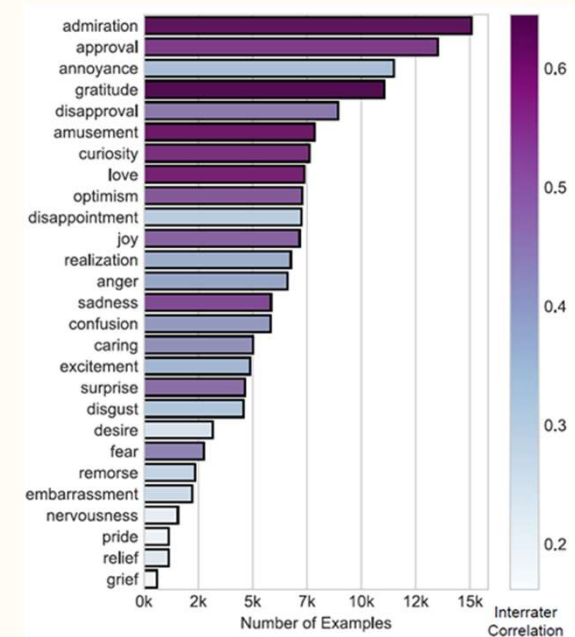
Dataset Viewer Auto-converted to Parquet API Embed Data Studio

Subset (2) Split (3)
simplified · 54.3k rows train · 43.4k rows

Search this dataset

text string · lengths 2~73 56.4%	labels sequence · lengths 1~2 83.6%	id string · lengths 7 100%
Damn youtube and outrage drama is super lucrative for reddit	[0]	ed8mbdn
It might be linked to the trust factor of your friend.	[27]	eczgvlo
Demographics? I don't know anybody under 35 who has cable tv.	[6]	ee16g5h
Aw... she'll probably come around eventually, I'm sure she was just jealous of [NAME]... I mean, what woman wouldn't be! lol	[1, 4]	edex4ki
Hello everyone. Im from Toronto as well. Can call and visit in personal if needed.	[27]	ef83m1s
R/sleeptrain Might be time for some sleep training. Take a look and try to feel out what's right for your family.	[5]	efh7xnk

< Previous 1 2 3 ... 435 Next >



- Used GoEmotins dataset (Google Research)
- Number of examples: 58,009
- Number of emotions: 27 + neutral

Dorottya Demszky, Dana Movshovitz-Attias, Jeongwoo Ko, Alan Cowen, Gaurav Nemade, and Sujith Ravi. 2020. *GoEmotions: A Dataset of Fine-Grained Emotions*. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4040–4054, Online. Association for Computational Linguistics. ([10.18653/v1/2020.acl-main.372](https://doi.org/10.18653/v1/2020.acl-main.372))

Feature Highlight #2 - Machine Learning

Remapping the Emotions

Happy

- Admiration
- Amusement
- Approval
- Caring
- Excitement
- Gratitude
- Joy
- Love
- Optimism
- Pride
- Relief

Sad

- Disappointment
- Grief
- Remorse
- Sadness

Angry

- Anger
- Annoyance
- Disapproval
- Disgust

Anxious

- Fear
- Nervousness
- Embarrassment

Curious

- Curiosity
- Desire

Surprised

- Surprise

Confused

- Confusion
- Realization

Neutral

- 27 emotions were regrouped into 7 emotions with neutral to capture the basic emotions and decrease the model size
- 7 was chosen because if more than 7, there were many emotion groups with one or two emotions, and the number of samples in these emotion groups are very small (imbalanced data)

Feature Highlight #2 - Machine Learning Model Training

- **Converted to multi-hot vector**
 - Emotion_classes = ['angry', 'sad', 'anxious', 'happy', 'curious', 'confused', 'surprised', 'neutral']
 - E.g. if the emotions are 'happy' and 'anxious': Multi-hot vector: [0, 0, 1, 1, 0, 0, 0, 0]
- **Tokenization**
 - Used DeBERTa tokenizer and tokenized all training, validation, and test texts with truncation at a maximum length of 64 tokens (4-6 sentences)
 - DeBERTa: transformer based model developed by Microsoft, widely used for a variety of NLP tasks
- **Class weights calculation**
 - Computed class weights to address class imbalance using the ratio of the samples to total samples
- **Model definition**
 - Used the pre-trained DeBERTa model for multi-label classification
 - 8-class output (7 emotions + neutral)
- **Training loop**
 - AdamW optimizer with a learning rate of $2.5e-5$ (optimized with small dataset)
 - Batch size = 16 (optimized with small dataset)
 - Epochs = 10, implemented early stopping based on macro F1 score improvement
 - Evaluated model performance on the validation set using fixed thresholds for multilabel predictions (threshold=0.5)
- **Threshold tuning**
 - Tuned per-class thresholds on the validation set to maximize F1 scores using a range of thresholds
 - Saved the best model based on F1 score
- **Fallback logic**
 - If no emotion exceeds the threshold, assign the top predicted emotion

Feature Highlight #2 - Machine Learning ML Model Evaluation

Evaluation on test set	
F1 Score (Micro)	0.658
F1 Score (Macro)	0.574
Hamming Accuracy	0.900

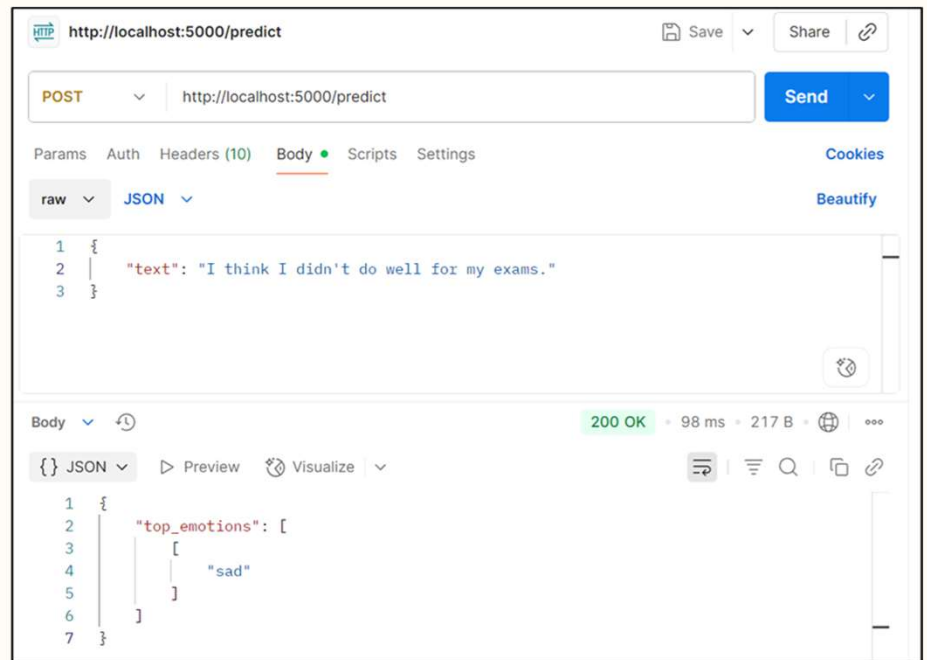
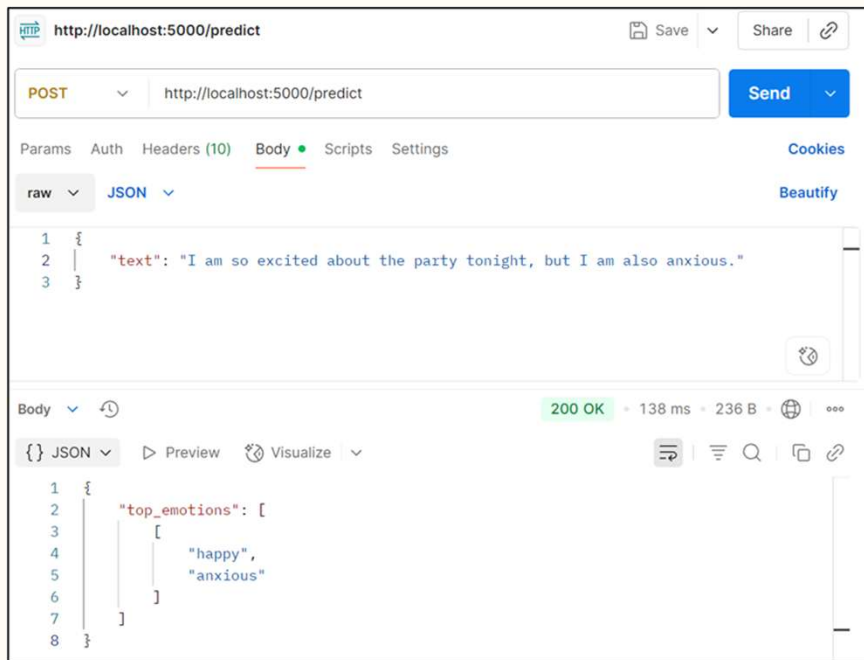
- For multi-label classification tasks, F1 scores of 0.6-0.7 (micro average) is typical, especially when the classes are imbalanced
- Hamming accuracy of 0.9 is strong, indicating that the model is correctly predicting most of the labels per instance

	Precision	Recall	F1 Score	Support
Angry	0.54	0.65	0.59	780
Sad	0.52	0.61	0.56	331
Anxious	0.53	0.64	0.58	128
Happy	0.77	0.84	0.80	1966
Curious	0.43	0.68	0.53	338
Confused	0.31	0.40	0.35	275
Surprised	0.51	0.57	0.54	136
Neutral	0.58	0.74	0.65	1787
Micro avg	0.60	0.73	0.66	5741
Macro avg	0.52	0.64	0.57	5741

- The macro F1-score is lower than micro because certain labels are much easier to predict (happy, neutral) than others (confused, curious)

Feature Highlight #2 - Machine Learning

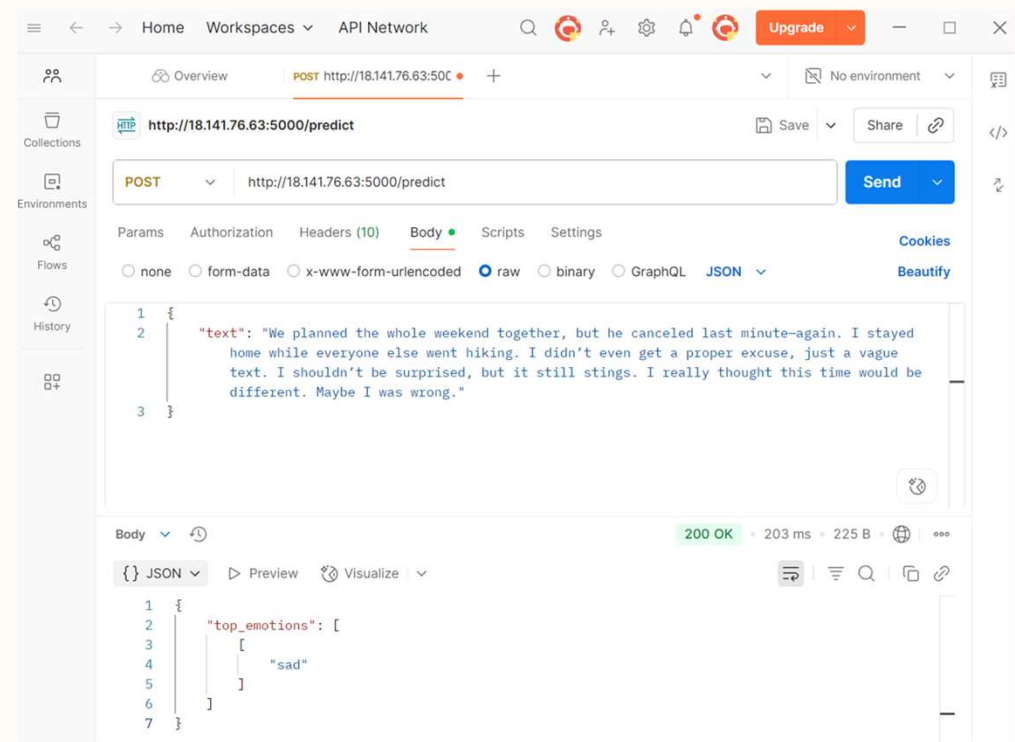
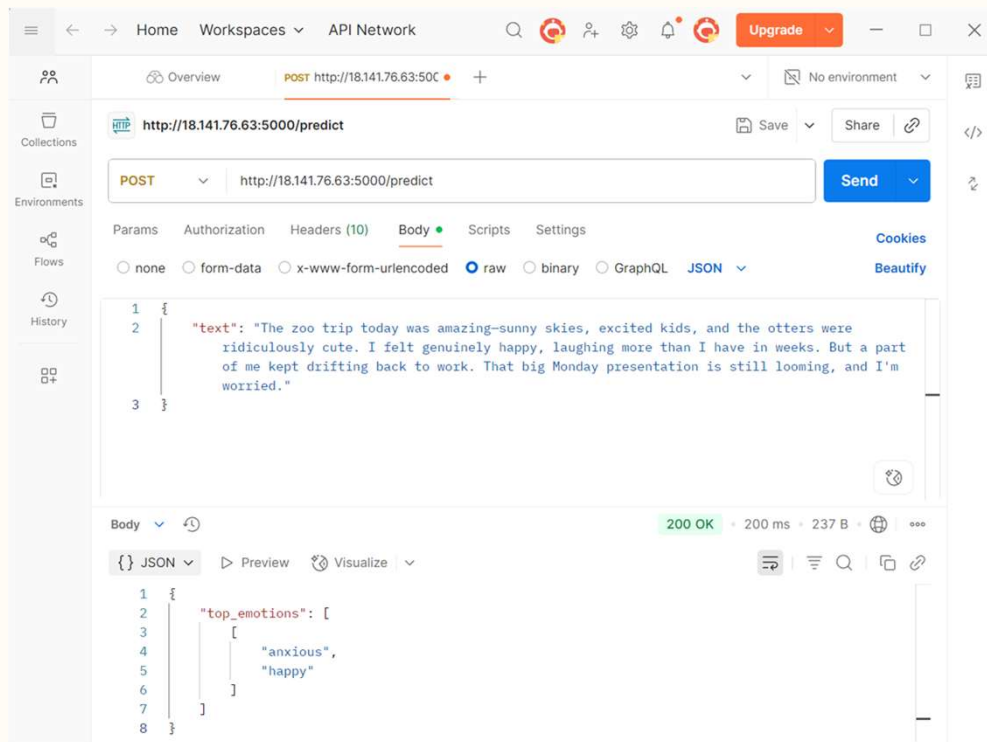
Checking the model



- The predictions of the emotions are reasonable

Feature Highlight #2 - Machine Learning

Checking the model with longer text



- The text above are close to 64 tokens (maximum length)
- The predictions of the emotions are reasonable

Feature Highlight #3 - Dashboards

The User Dashboard required 3 projections of data:

1. Journal data averaged by day
1. Total count of emotions
1. Habits data by day

The Counsellor Dashboard doesn't need Habits, but needs the "Clients" they are linked with.

On the frontend, during render, a data hook is used to call the API from the backend, then sets the data in state. The parent dashboard page pulls from the hook, and passes as props to the various child components. The component with the date picker display the value via prop, and onChange, calls the parent to setState, and re-render

Appendix

Problem statement sources:

https://www.imh.com.sg/Newsroom/News-Releases/Documents/NYMHS_Press%20Release_FINAL19Sep2024.pdf

<https://www.channelnewsasia.com/singapore/poor-mental-health-young-adults-seek-help-moh-survey-3802531>

[Efficacy of journaling in the management of mental illness: a systematic review and meta-analysis](#)